Enzyme Module:

Summary of developing the first external module to the $MLN – protocol

- by A$H and Willie Laubenheimer @iemwill aka flex-IT

Introduction

In the middle of 2019, I joined the Team of Midas Technologies AG as Distributed Ledger Technology developer, to work on their Milestones granted by the former Melon Council (rebrand to Enzyme) building the first external modules for their decentralized fund management protocol.

The backend of the protocol is written in solidity with a typescript framework. It is the first of its kind and enables decentralized management of ERC20-token-funds with fixed rules and open-source code.

In 2020 the melon project published their new structure of the protocol and changed the publishing brand to Enzyme. First, we need to check if a go through of my development process still makes sense in a way that the build modules are adaptable to the new core protocol and maybe even to the new repository.

Background Information

The following is a comparison between the first and second repository containing the core protocol of the melon/enzyme decentralized fund management system ($MLN).

My first stop was https://github.com/melonproject where have been several projects as well as the protocol repository. As one can see by now the Organization is empty since it has rebranded and moved with the development to Enzyme under Avantgarde Finance. (You can read more about that in the following articles: Announcement of future rebranding and the reason https://medium.com/enzymefinance/from-melon-to-watermelon-57e8feb02f0f, v1.0 release Article https://medium.com/enzymefinance/melon-v1-0-zahreddino-60105f51988d, closure of the first development stage https://medium.com/enzymefinance/goodbye-melonport-hello-world-a64b5d116b13, introducing Avantgarde Finance https://medium.com/enzymefinance/avantgarde-finance-awarded-lead-developer-role-to-fulfil-the-next-3-year-roadmap-on-melon-7489bf167375 & introducing Enzyme https://medium.com/enzymefinance/from-melon-to-enzyme-b5b56512f40d).

The repository I was working on can be found at https://github.com/Midas-Technologies-AG/protocol/tree/bbc8175016b15c0b48599e4cdbad5262fe1d460a the last commit from my past work on the Melon/Giveth Module which enables donations from a decentralized fund to a decentralized DAC/NGO.

The other two modules I developed, a copy fund module and a multimanager module, can be found here https://github.com/Midas-Technologies-

AG/MelonApp/tree/1e65c2b9a4c24b193a5366f2acc387a5f85e6e0a. If the links don't exist change Midas-Technologies-AG to iemwill and remove everything after 'MelonApp'. That will point to the same repositories on my GitHub.

The latest repository of the now called Enzyme protocol can be found at https://github.com/enzymefinance/protocol and is at version 4.

Brief Overview of Changes

There have been a lot of minor and some major changes from version v1 to v4. As a developer, the first thing which comes to mind is having a look at the folder structure, which I will visualize in the following:

V1	V4
compile	audits
deploy	contracts
deployments	lib
src	packages
tests	tests/persistent
thirdParty	

While in v1 every contract and the core blockchain protocol is in the 'src' directory listed in their matching part of the protocol structure, v4 contains all solidity files in the contracts directory and furthermore after a first and short go through one sees that there is a completely new structure.

According to some articles from the Enzyme Blog on Medium, such as Redesigning Melon/Enzyme https://medium.com/enzymefinance/redesigning-melon-enzyme-6ccb9853bf83, upgrade to Enzyme Sulu https://medium.com/enzymefinance/enzymes-sulu-is-live-62721f3b2bb8 and a new fee construction process tackling also Ethereum Gas fees https://medium.com/enzymefinance/this-new-enzyme-feature-will-obliterate-your-ethereum-gas-fee-concerns-9f6ff20477ed, the new repository structure reflects those changes.

Module Adoption Possibilities

Since the Giveth-Module is an external module, one sees that the v1 handled those via third parties and in v4 the analogous can be found in 'protocol/contracts/release/extensions'. After a brief overview it seems to be very likely that the Giveth-Module doesn't need core changes to be adapted in v4 of the protocol. Of course there need to be made some adjustments to fit in the new version of the protocol, which most likely will be in 'protocol/contracts/release/extensions/integration-manager'. The toolset is still in typescript and the functions would be in 'protocol/packages/protocol/src/utils/integrations'. Also, the two other modules should be possible to adapt, with adjustments of course.

Built Modules

Melon <-> Giveth

The first module should enable transactions from a fund to an external DAC (Decentralized Altruistic Community) from Giveth (https://giveth.io/). Since funds in the protocol were (and still are) hold by a Vault Contract and a secure delegation process, there were several solidity lines to be written, as well as Typescript functions for general integration and to be useable by its users. However, the main part was to add a trading function to an external exchange, the giveth bridge, which then delegates to a specific giveth-DAC.

A brief overview of the results can be read here https://medium.com/ash-blog/ash-milestone-report-1-84be6f41a3fd in the Section 'Update on current development work'.

The core part was to create an exchange adapter which follows the given rules of the core protocol, its delegation process and an approval for the giveth bridge to transfer the amount of token one wants to donate. To transfer the token towards the receiver DAC, a call from the fund-owner via 'makeOrder' to the giveth bridge needed to be approved. Since it wasn't really an order one makes my first suggestion was to also call it different and can be found here: https://github.com/Midas-Technologies-AG/protocol/blob/donateOnExchange/src/contracts/exchanges/adapters/GivethBridgeAdapter.sol.

The Typescript file to use the solidity contract can be found here https://github.com/Midas-Technologies-AG/protocol/blob/donateOnExchange/src/contracts/exchanges/transactions/donateViaGivethBridgeAdapter.ts.

For ETH donations one doesn't needed the adapter and the function working in test environments can be found here https://github.com/Midas-Technologies-AG/protocol/blob/donateOnExchange/src/contracts/exchanges/transactions/sendGivethETH.ts, or use the adapter function with the asset 0x0.

However, to fit into the ecosystem of the protocol and the surrounding checks the following solution was delivered to Melon/Enzyme https://github.com/Midas-Technologies-AG/protocol/blob/givethCallOnExchange/src/contracts/exchanges/adapters/GivethBridgeAdapter.sol. Since there was not much communication, I was not sure how they prefer it to be done, so I did several solutions.

To summarize the work, it was difficult to get into the system of the protocol (backend-side) on my very own and after some failures I managed to create a running prototype tested on test networks Ropsten and Kovan, as well as internal test environments. Now, after more than 2 years I see a lot of optimization possibilities and unnecessary things, which I may remove on my own repo. Since the interest wasn't big to continue that module (maybe also caused by the pandemic), I guess my motivation wasn't neither. Maybe new motivation will rise to continue and take it live.

The following modules were built after the Giveth Module, which needed an addition to the core protocol and took me some time to develop, since it is a very complex and sensitive protocol. None of the following modules needed integration in the core protocol, which make their complexity a lot easier than the Giveth Module.

Copy Funds

The copy fund module is self-describing. It copies all assets of a given fund and buys them proportional to their size in the given fund with the proportional amount one is willing to invest. To use this function one gives an amount in wrapped Ether (WETH) and the address of the fund one wants to copy. Since we need to remember that we copied a fund, or we already sold that copied fund, I created a contract to log all actions. There is also a function in this module to sell a previous copied fund.

If you are interested in the code checkout this link Midas-Technologies-AG/MelonApp/copyFundModule.js. Another reminder, that all functions from those Modules are not tested “live” yet and are in proof-of-concept stage, so don’t use them with any “real” token.

Multimanager Funds

The idea of this module is to setup a fund not managed by a single entity, rather than by several entities. For a proper use of all core protocol functions by several fund managers, the following protocol functions were additionally coded for a multimanager fund:

multiSigAddOwner, confirmTx, executeTx, beginSetupMSW, completeSetupMSW, makeOrderMSW, takeOrderMSW, cancelOrderMSW, returnAssetToVaultMSW, redeemMSW

Setting up a fund needs all requirements of the multisignature-wallet, as well as every transaction executed by the fund. Let’s say the MSW has five manager and needs, for every transaction execution, at least 3 confirmations. If we want to setup the fund, we execute the ‘beginSetupMSW’ function by one of the five addresses from the MSW and to get the transaction proposed to the network we need at least 2 more confirmations of our (internal) proposed transaction. To make orders as well as to take or cancel orders for an exchange, the procedure stays the same.

If you are interested in the code checkout the following link Midas-Technologies-AG/MelonApp/multiManagerModule.js. There are also lines to the proof of concept and some files which help you to try it yourself.

Conclusion

It took me a lot of experimenting and research to dig deeper into the core protocol and the corresponding workaround, especially because I was completely on my own and did not have developing backup or introductions by the melon/enzyme dev team. After some failures I managed to build the first external adapter for the first decentralized fund management protocol (Juhuuu😊. Since it wasn't planned to bring the module on a live stage, I created a proof of concept with working examples on test net as well as reproduceable code. The other two modules were also delivered in time, with reproduceable code.

Shortly after delivery the rebranding of Melon happened, and a completely new work of their repository and some core functions changed. To get the done work on a live stage some adjustments of the delivered modules according to the changes of the protocol (from v1 - v4) need to be made.

Since the pandemic hit almost every sector, a further funding to the A$H app from Midas Technologies AG didn't happen, which stopped the development of the A$H app as well as these first external modules. If there is any interest to work on the modules, enhanced or otherwise related ones, contact me via https://Laubenheimer.eu and let me know what you are up to.

Finally, it was a pleasure for me to work on such an interesting and complex protocol, as well as to get hands on experience in the world of blockchains and decentralisation. Kudos to the whole team working on that protocol.

Donations

Bitcoin: 3QHgobkrPPyz6SJe3WVwZvazj5e1u4RVcf

Ethereum(EVM): 0x7F0EFFEe4bf10B3e3E5BdA022289508f367c7eDa

Interesting Links

Summary of MLN token economics: https://medium.com/enzymefinance/everything-you-always-wanted-to-know-about-melon-but-were-too-afraid-to-ask-9ea3fda6d71f

Melon (Enzyme) Council: https://medium.com/enzymefinance/melon-council-unveiled-at-m-1-ae87d999b7ba

Decentralized Government: https://medium.com/enzymefinance/melon-will-run-its-decentralized-governance-on-aragon-9f7935693720

Motivation, Innovation and Inspiration: https://medium.com/enzymefinance/monetising-your-vaults-the-inner-workings-of-fees-on-enzyme-d5b275e7b9f5

Token Update: https://medium.com/enzymefinance/the-state-of-enzymes-tokenomics-9ff46aecf160